SearchLens: A Reproducible Classical Search Engine with Positional Indexing, BM25 Ranking, and Pseudo-Relevance Feedback

Student Name
CECS 429/529: Search Engine Technology
California State University, Long Beach, USA
student@example.edu

Abstract

This project implements SearchLens, a compact but complete lexical search engine designed for reproducible experimentation in a search-engine technology course. The system builds a positional inverted index over a 60-document synthetic technical corpus, supports Boolean and phrase retrieval, ranks candidate documents with TF-IDF cosine similarity and BM25, and optionally applies pseudo-relevance feedback. The evaluation uses 12 query topics and 75 graded relevance judgments with Precision@5, Recall@10, MAP, MRR, and nDCG@10. BM25 achieved the strongest MAP (0.8276) and nDCG@10 (0.8851), while BM25 with feedback achieved the highest Recall@10 (0.8641) but slightly lower precision because of topic drift. The main contribution is an inspectable end-to-end implementation that connects core information retrieval theory to runnable code, experiments, a local web demo, and a conference-style report.

Keywords

information retrieval, search engine, inverted index, BM25, TF-IDF, pseudo relevance feedback, evaluation

1 Introduction

Search engines are often introduced as large industrial systems, but their essential mechanisms can be studied in a compact implementation: text normalization, inverted indexing, candidate generation, ranking, and evaluation. SearchLens was built to make those components explicit and reproducible. Instead of depending on a black-box search library, the implementation stores positional postings, computes document statistics, evaluates multiple ranking functions, and exposes both a command-line interface and a small web demo.

The project addresses the following problem: given a small collection of technical documents, can a transparent classical retrieval engine rank relevant documents effectively while preserving enough internal state to explain its results? This problem is appropriate for a search-engine technology course because it requires connecting data structures, algorithms, models, and evaluation methodology.

The contributions are fourfold. First, SearchLens implements a full lexical retrieval pipeline using only the Python standard library. Second, it compares Boolean-overlap ranking, TF-IDF cosine similarity, BM25, and BM25 with conservative pseudo-relevance feedback. Third, it includes a reproducible test collection with documents, queries, graded judgments, saved index files, metrics, and unit tests. Fourth, it packages a local web interface and Docker deployment path so the system can be demonstrated live.

Target venue. Target venue for this project: **ACM SIGIR Conference on Research and Development in Information Retrieval**. SIGIR is suitable because the project concerns search, ranking, relevance feedback, and experimental evaluation, all central to information retrieval research and system demonstrations [1]. The report uses the ACM `acmart` conference style as a formal submission format [2].

2 Related Work

Classical information retrieval is grounded in the inverted index and ranked retrieval models summarized by Manning, Raghavan, and Schuetze [5]. Their textbook describes tokenization, term vocabularies, postings lists, phrase queries, vector-space scoring, BM25, relevance feedback, and evaluation. SearchLens differs from a textbook presentation by implementing these components in a runnable project with test data, command-line experiments, unit tests, and a demonstration interface.

The vector space model introduced by Salton, Wong, and Yang represents documents and queries as term-weighted vectors and compares them by similarity [8]. SearchLens implements this idea with TF-IDF weights and cosine normalization. This model is interpretable and useful as a baseline, but it does not include explicit term-frequency saturation or document-length normalization.

BM25, developed from probabilistic retrieval models, is a strong lexical baseline because it combines inverse document frequency, term-frequency saturation, and document-length normalization [6]. SearchLens implements BM25 directly over the positional inverted index and compares it against TF-IDF and Boolean-overlap ranking.

Relevance feedback was popularized through the SMART system and the Rocchio algorithm, which modifies a query vector using evidence from relevant and nonrelevant documents [7]. SearchLens implements a lightweight pseudo-relevance feedback variant: it assumes the first-pass top BM25 results are useful, extracts high-idf terms from those results, and appends at most three expansion terms. This design is intentionally conservative because feedback can also cause topic drift.

Web-scale systems add link analysis, crawling, spam resistance, and distributed indexing. PageRank is a classic example of using the web graph as an authority signal [3]. SearchLens does not implement a crawler or link-based ranking; it focuses on the lexical core, which is still necessary for modern hybrid search systems.

Corpus JSONL → preprocessing → positional inverted index → candidate generation → ranking model → snippets, metrics, or web results

Figure 1: SearchLens architecture. The implementation shares one retrieval core across the CLI, experiments, tests, and web demo.

3 Methods

3.1 System Architecture

Figure 1 shows the SearchLens pipeline. A JSONL corpus is loaded into memory, normalized, and indexed. Query processing first parses phrases and query terms, generates candidates from postings lists, filters phrase constraints, ranks candidate documents, and returns snippets with term highlighting. The same retrieval code is used by the command-line tool, evaluation script, unit tests, and web demo.

3.2 Data Processing Pipeline

Each document contains a document identifier, title, body, category, tags, and optional URL. The indexer concatenates title and body, lowercases text, tokenizes alphanumeric terms, removes a small stopword list, and applies light suffix normalization. The normalizer is deliberately simple so the preprocessing decisions can be inspected and modified.

The positional inverted index stores two forms of document text. For scoring, it stores stopword-filtered normalized terms and term frequencies. For phrase matching, it stores normalized token sequences that preserve stopwords, allowing quoted phrases to be checked as consecutive token spans. For every term t, the postings structure stores a mapping from document identifier d to a sorted list of positions. The index also stores document length, average document length, document tokens, and vocabulary statistics.

3.3 Retrieval Models

Boolean-overlap baseline. The simplest ranking baseline counts how many unique query terms occur in a candidate document. This is not a pure Boolean set output; it is a Boolean-style overlap score so the method can be compared with ranked models using the same metrics. The implementation also provides a separate Boolean expression parser supporting `AND`, `OR`, `NOT`, parentheses, and operator precedence.

TF-IDF cosine ranking. For TF-IDF, each term weight is

$$w_{t,d} = (1 + \log tf_{t,d}) \cdot idf_t, \tag{1}$$

where idf_t is computed with the same smoothed inverse document frequency routine used by the ranking module. The score is the cosine similarity between query and document vectors:

$$score(q, d) = \frac{\sum_{t \in q} w_{t,q} w_{t,d}}{\|q\| \|d\|}. \tag{2}$$

BM25 ranking. BM25 uses term-frequency saturation and document-length normalization. SearchLens uses the following scoring function:

$$score(q, d) = \sum_{t \in q} idf_t \cdot \frac{tf_{t,d}(k_1 + 1)}{tf_{t,d} + k_1(1 - b + b|d|/avgdl)}, \tag{3}$$

with $k_1 = 1.5$ and $b = 0.75$. The inverse document frequency is

$$idf_t = \log\left(1 + \frac{N - df_t + 0.5}{df_t + 0.5}\right), \tag{4}$$

where N is the number of documents and df_t is document frequency.

Pseudo-relevance feedback. The BM25+PRF configuration first runs BM25, takes the top three documents as pseudo-relevant, scores candidate expansion terms by discounted TF-IDF, removes original query terms, and appends the top three terms. The query is then run again with BM25. This design trades recall for a controlled risk of topic drift.

3.4 Implementation and Reproducibility

The code is organized into four core modules: `preprocess.py`, `index.py`, `search.py`, and `evaluate.py`. Reproducibility scripts rebuild the dataset, index, metrics, and sample results. The project includes five unit tests covering index statistics, BM25 ranking, phrase matching, Boolean parsing, and pseudo-relevance feedback. A minimal web demo uses Python's `http.server`, so no external web framework is required.

4 Experiments

4.1 Dataset

The experiment uses a self-contained synthetic test collection named SETC-60. It contains 60 short technical documents across 12 topics: indexing, Boolean retrieval, BM25, TF-IDF, crawling, PageRank, spelling correction, query expansion, evaluation, compression, neural/hybrid retrieval, and robustness. The query set contains 12 user-style topics, one per category. The relevance file contains 75 graded judgments: grade 2 for primary-topic relevance and grade 1 for related documents that are intentionally useful but not central.

The synthetic design is a limitation, but it gives three advantages for this project. First, the entire corpus can be inspected quickly. Second, each query has known relevant documents, allowing deterministic metric computation. Third, the dataset covers many concepts from search-engine technology, enabling a broad implementation without external downloads.

4.2 Experimental Setup

All methods were evaluated using the same preprocessing, candidate generation, and top-10 cutoff. The aggregate metrics were Precision@5, Recall@10, mean average precision (MAP), mean reciprocal rank (MRR), and nDCG@10. nDCG is included because the judgments are graded; it rewards placing grade-2 documents above grade-1 documents [4]. The experiments are reproduced by running `PYTHONPATH=src python scripts/run_experiments.py`.

Table 1: Aggregate retrieval metrics over 12 queries. Higher is better.

Method	P@5	R@10	MAP	MRR	nDCG@10
Boolean overlap	0.8333	0.8216	0.7736	1.0000	0.8487
TF-IDF cosine	0.8667	0.8593	0.8242	1.0000	0.8842
BM25	**0.8833**	0.8593	**0.8276**	1.0000	**0.8851**
BM25 + PRF	0.8667	**0.8641**	0.8166	1.0000	0.8815

4.3 Results

Table 1 reports the aggregate results. BM25 achieved the best Precision@5, MAP, and nDCG@10. TF-IDF was close to BM25, which is expected because the corpus is short and topical. BM25+PRF achieved the best Recall@10 but slightly reduced Precision@5 and MAP, suggesting that feedback introduced some related but lower-grade documents into early ranks.

4.4 Analysis

The Boolean-overlap baseline performed reasonably because the synthetic topics use discriminative technical terms. However, it lacks principled term weighting and therefore ranked some broad documents too high. TF-IDF improved over Boolean-overlap by emphasizing specific terms and normalizing vector length. BM25 improved slightly further by saturating repeated term frequency and adjusting for document length. This result supports the common practice of treating BM25 as a strong lexical baseline.

The feedback result is more nuanced. Recall@10 increased from 0.8593 to 0.8641, but MAP decreased from 0.8276 to 0.8166. Inspecting sample results showed that expansion terms sometimes retrieved related grade-1 documents earlier, which helped recall but displaced grade-2 documents. This is an expected failure mode of pseudo-relevance feedback when the first-pass top documents are not perfectly centered on the user's intent.

MRR was 1.0000 for every method because each query had at least one clearly matching relevant document at rank 1. This is useful for verifying the implementation but not sufficient for comparing systems. MAP and nDCG@10 are more informative in this setting because they account for multiple relevant documents and graded relevance.

4.5 Limitations and Threats to Validity

The main limitation is that the corpus is synthetic and small. The results demonstrate that the implementation works and that the ranking methods behave as expected, but they should not be interpreted as evidence of production-scale search quality. The relevance judgments are derived from topic labels with manual related-document additions, so they are less realistic than independent human assessments. The system also omits distributed indexing, crawling, sophisticated stemming, learned ranking, dense embeddings, and latency benchmarking. Future experiments should use a standard public test collection and include statistical significance testing.

5 Conclusion and Future Work

SearchLens completes an end-to-end classical search engine project: it ingests a corpus, builds a positional inverted index, supports Boolean and phrase retrieval, ranks with TF-IDF and BM25, applies pseudo-relevance feedback, evaluates results with standard metrics, and provides a web demo. The experiments show that BM25 is the strongest overall method on the SETC-60 test collection, while feedback slightly improves recall at the cost of precision and MAP.

Future work should extend SearchLens in four directions. First, replace or augment the synthetic collection with a public benchmark such as Cranfield, CACM, or a TREC-style subset. Second, add index compression with gap encoding, variable byte codes, and skip pointers. Third, add a hybrid dense retrieval component to compare lexical and semantic matching. Fourth, deploy the web demo publicly and collect interactive feedback to evaluate usability in addition to offline metrics.

Acknowledgments

This project was prepared for CECS 429/529: Search Engine Technology. Replace the placeholder author information before submission.

References

[1] ACM SIGIR. 2026. ACM SIGIR Conference on Research and Development in Information Retrieval. https://sigir2026.org/en-AU/pages/submissions/full-papers-track. Accessed May 6, 2026.

[2] Association for Computing Machinery. 2025. The ACM Article Template: Using LaTeX. https://authors.acm.org/proceedings/production-information/preparing-your-article-with-latex. Accessed May 6, 2026.

[3] Sergey Brin and Lawrence Page. 1998. The Anatomy of a Large-Scale Hypertextual Web Search Engine. *Computer Networks and ISDN Systems* 30, 1–7 (1998), 107–117. doi:10.1016/S0169-7552(98)00110-X

[4] Kalervo Jaervelin and Jaana Kekaelaeinen. 2002. Cumulated Gain-Based Evaluation of IR Techniques. *ACM Transactions on Information Systems* 20, 4 (2002), 422–446. doi:10.1145/582415.582418

[5] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schuetze. 2008. *Introduction to Information Retrieval*. Cambridge University Press. https://nlp.stanford.edu/IR-book/

[6] Stephen E. Robertson and Steve Walker. 1994. Some Simple Effective Approximations to the 2-Poisson Model for Probabilistic Weighted Retrieval. In *Proceedings of the 17th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM/Springer, 232–241. doi:10.1007/978-1-4471-2099-5_24

[7] J. J. Rocchio. 1971. Relevance Feedback in Information Retrieval. In *The SMART Retrieval System: Experiments in Automatic Document Processing*. Prentice-Hall, Englewood Cliffs, NJ, 313–323.

[8] Gerard Salton, A. Wong, and C. S. Yang. 1975. A Vector Space Model for Automatic Indexing. *Commun. ACM* 18, 11 (1975), 613–620. doi:10.1145/361219.361220